Nixie Clock Ultra

Systemdokumentation

Version 2.1 · ESP32-S3 · 6 × IN-12A Nixie
MCP23017 Direct Drive · DS1302 RTC · WS2812B NeoPixel

Stand: Juli 2026
Projekt: A. Schröder

Systemübersicht

Die Nixie Clock Ultra ist eine Röhrenuhr auf Basis des ESP32-S3 Mikrocontrollers mit 6 IN-12A Nixie-Röhren. Das System besteht aus zwei Platinen: dem Logic Board, das den Mikrocontroller, die Echtzeituhr, die Hochspannungserzeugung und alle Schnittstellen beherbergt, sowie dem Nixie Display Board, das die Anzeigeelektronik mit vier MCP23017 I²C-Port-Expandern, 60 NPN-Transistoren und den WS2812B RGB-LEDs enthält.

Die Röhren werden ohne Multiplexing direkt angesteuert — jede der 60 Kathoden besitzt einen eigenen SMBTA42-Hochvolt-Transistor. Ghosting-Effekte sind dadurch vollständig eliminiert.

Zweiplatinenaufbau

Platine	KiCAD-Projekt	Rev.	Hauptfunktion
Logic Board	nixieclocklogic_V2-2	2.1	ESP32-S3, DS1302 RTC, DC-DC 5V→10V, HV-MOD, LDR, Taster, IR, USB
Nixie Display Board	nixieclockin12_V2-1	2.01	4× MCP23017, 60× SMBTA42, 6× IN-12A, WS2812B, optional 6× TLP627

Inter-Board-Steckverbinder

Die beiden Platinen sind über zwei Steckverbinder verbunden. J3 (Logic Board) ↔ J1 (Display Board) führt Logik- und Steuersignale, J4 (Logic Board) ↔ J2 (Display Board) die Hochspannungsversorgung.

Steckverbinder	Polig	Signale
J3 ↔ J1 (Logic)	8	VCC 5 V (×2), SCL (GPIO9), SDA (GPIO8), frei, NEO-Daten (GPIO21), GND (×2)
J4 ↔ J2 (HV)	4	HV ~170V DC (×2), HVgnd (×2)

PCB 1 – Logic Board

Das Logic Board (Rev 2.1, KiCAD: nixieclocklogic_V2-1) trägt alle zentralen Komponenten des Systems: den ESP32-S3-WROOM-1 Mikrocontroller (U1), die DS1302 Echtzeituhr (U2), den VS1838B IR-Empfänger (U3), das HV-Boost-Modul (U4, 10 V → ~170 V), den AMS1117-3.3 LDO-Regler (U5, 5 V → 3,3 V), den DC-DC-Boost-Konverter (U6, 5 V → 10 V), den USB-ESD-Schutz (U22), einen LDR-Helligkeitssensor (J5), vier Taster sowie den Micro-USB-Anschluss (J2).

Gegenüber Rev 2.0 wurden zwei Änderungen vorgenommen: Das HV-Modul benötigt mindestens 10 V Eingangsspannung, um stabile 170 V für die IN-12A-Röhren zu liefern — direkt aus USB-5 V war dies nicht erreichbar. Daher wurde ein DC-DC-Konverter (5 V → 10 V) ergänzt. Außerdem wurde der LDR-Helligkeitssensor (J5) neu bestückt, der in Rev 2.0 noch nicht vorhanden war.

Schaltplan

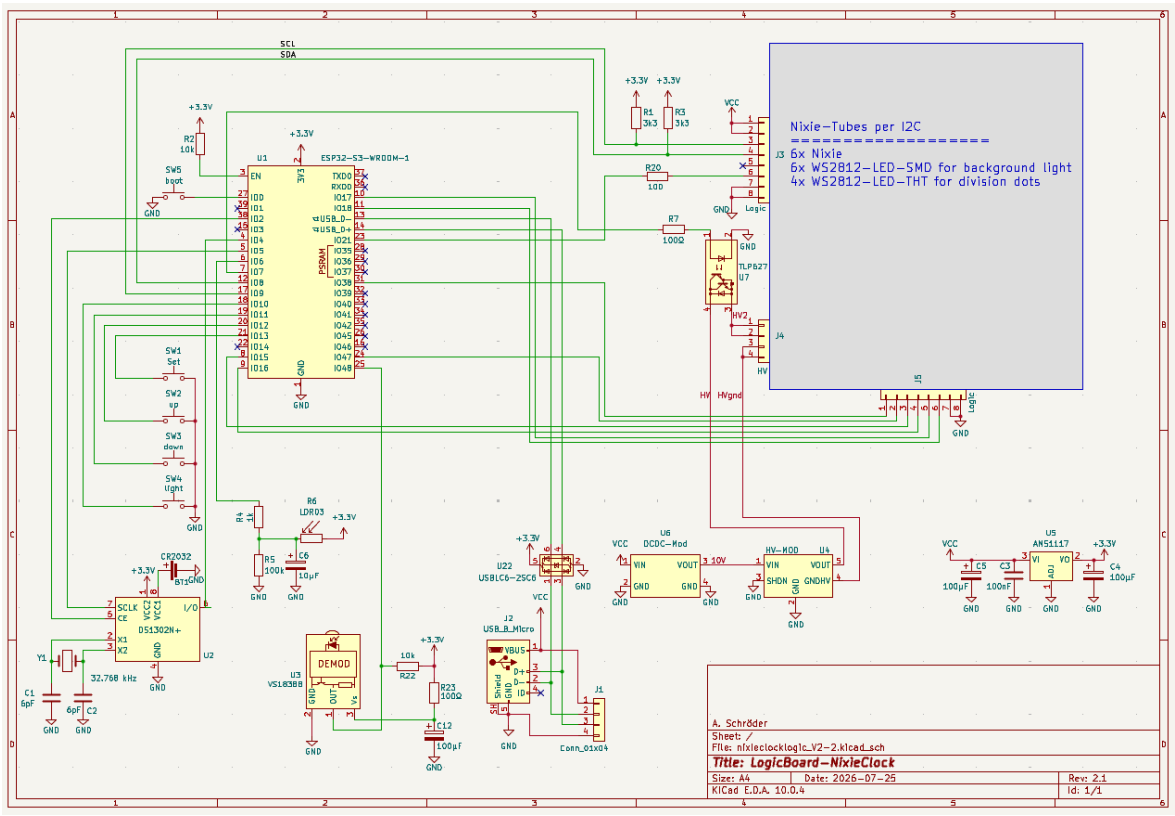


Abb. 1 – Logic Board Rev 2.1 Schaltplan (nixieclocklogic_V2-1)

PCB-Layout

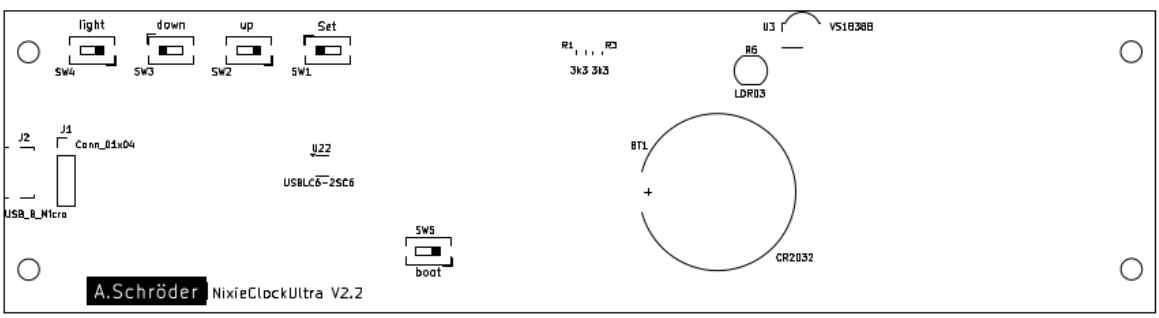


Abb. 2a – Logic

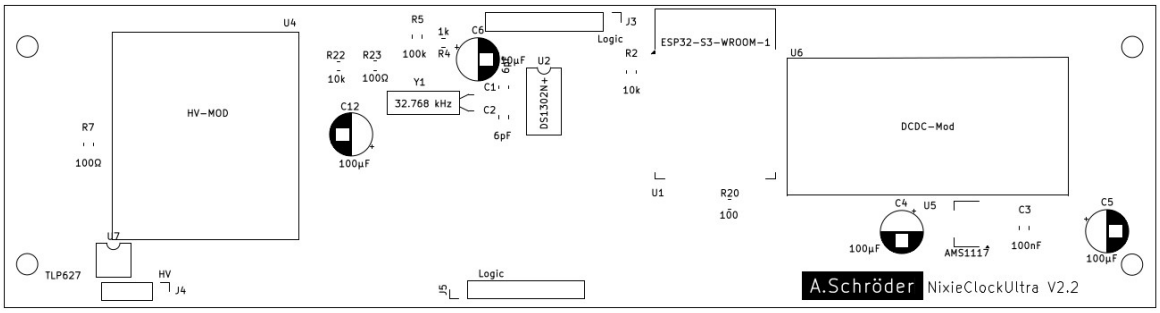


Abb. 2b – Logic Board Rev 2.1 PCB-Layout Unterseite

Komponenten

Ref.	Bauteil	Gehäuse	Funktion
U1	ESP32-S3-WROOM-1	Modul	Mikrocontroller, WiFi/BT, 4 MB Flash
U2	DS1302N+	DIP-8	Batteriegepufferte Echtzeituhr (ThreeWire)
U3	VS1838B	TO-92	IR-Empfänger/-Demodulator 38 kHz
U4	HV-MOD	Custom	Boost-Converter 10 V → ~170 V DC für Nixie-Anoden
U5	AMS1117-3.3	SOT-223	LDO-Linearregler 5 V → 3,3 V
U6	DC-DC-MOD	Custom	Boost-Konverter 5 V → 10 V (Vorstufe für HV-Modul)
U7	TLP627	DIP-4	Optokoppler schaltet Anodenspannung per Hardware-PWM (Nacht-Modus-Dimmung)
U22	USBLC6-2SC6	SOT-23-6	USB-ESD-/TVS-Schutz
BT1	CR2032	SMD-Halter	RTC-Backup-Batterie (~3 V)
Y1	32,768 kHz Quarz	Radial	RTC-Taktquelle
SW1–4	Taster SET/UP/DOWN/LIGHT	SMD	Bedienpanel
SW5	BOOT	SMD	ESP32-Bootmodus (Programmierung)
J2	USB Micro-B	Custom	5 V-Einspeisung & Firmware-Upload
J3	PinHeader 1×8	2,54 mm	Inter-Board Logik-Signale
J4	PinHeader 1×4	2,54 mm	Inter-Board HV-Versorgung
J5	PinHeader 1×2	2,54 mm	LDR-Helligkeitssensor (neu in Rev 2.1): LDR→VCC, 100 kΩ→GND, Messung GPIO6

Spannungsversorgungskonzept

Die Versorgung erfolgt über den Micro-USB-Anschluss J2 (5 V). Aus der 5-V-Schiene erzeugen drei Wandler die benötigten Spannungen:

3,3-V-Logikversorgung (AMS1117-3.3)

Der AMS1117-3.3 (U5) ist ein Low-Dropout-Regler im SOT-223-Gehäuse. Er versorgt den ESP32-S3, den

DS1302, den IR-Empfänger sowie alle Pull-up-Widerstände. Abblockkondensatoren (100 nF + 100 µF) stabilisieren die Ausgangsspannung.

10-V-Zwischenstufe (DC-DC-MOD)

Das HV-Boost-Modul (U4) benötigt eine Eingangsspannung von mindestens 10 V, um zuverlässig ~170 V für die IN-12A-Röhren zu erzeugen — die USB-Einspannung von 5 V reicht dafür nicht aus. Der DC-DC-Konverter (U6) hebt die 5-V-USB-Schiene auf stabile 10 V an. Diese 10-V-Schiene speist ausschließlich das HV-Modul.

Hochspannung ~170 V DC (HV-MOD)

Der HV-MOD (U4) ist ein Boost-Converter-Modul, das aus 10 V eine geregelte Gleichspannung von ca. 170–180 V erzeugt. Diese Spannung wird auf dem Logic Board über einen TLP627-Optokoppler geführt, bevor sie über J4 an die Anoden aller sechs Nixie-Röhren gelangt. Die Kathoden der jeweils anzuzeigenden Ziffer werden durch NPN-Transistoren auf dem Display Board auf GND gezogen.

Anoden-Dimmung (TLP627, Hardware-PWM)

Der TLP627-Optokoppler (U7) schaltet die vom HV-MOD kommende Anodenspannung per Hardware-PWM (LEDC, 100 Hz) auf GPIO7 (HV_SWITCH_PIN). Im Nacht-Modus reduziert die Firmware den Duty-Cycle auf einen einstellbaren Wert zwischen 2 % und 60 % (Standard 25 %, siehe hv_dimmer.ino in firmware.md) statt wie zuvor die Kathoden per Software-PWM zu takten.

Optional lässt sich dieser eine Schalter durch 6 separate TLP627 (U5–U10) auf dem Nixie Display Board ersetzen, einen pro Röhre (siehe Kapitel «PCB 2 – Nixie Display Board»). Das ermöglicht einen weichen Ziffernwechsel pro Röhre, bei dem unveränderte Ziffern nicht mitblinken. Aktiviert wird das per HV_PER_TUBE_DIMMER-Compile-Switch in der Firmware (siehe firmware.md) — ohne bestückte Zusatzschalter bleibt der gemeinsame Schalter U7 aktiv. Der gemeinsame Schalter sollte bei dieser Umrüstung entfernt oder dauerhaft überbrückt werden; die Firmware hält sein Steuersignal (GPIO7) bei aktiviertem HV_PER_TUBE_DIMMER zur Sicherheit dauerhaft auf HIGH (offen), falls er nicht ausgebaut wird.

> ⚠️ Hochspannungswarnung: Der HV-Ausgang führt ~170 V DC — lebensgefährlich bei Berührung. Vor Arbeiten an der Schaltung die Versorgung trennen und Kondensatoren entladen.

ESP32-S3 GPIO-Belegung

GPIO	Signal	Richtung	Funktion
2	RTC_CE	Output	DS1302 Chip Enable (ThreeWire)
4	RTC_IO	Bi-Dir.	DS1302 Datenleitung (ThreeWire)
5	RTC_CLK	Output	DS1302 Takt (ThreeWire)
6	LDR_ADC	Input	LDR-Helligkeitssensor (ADC1, LDR → VCC, 100 kΩ → GND)
7	HV_SWITCH	Output	TLP627-Optokoppler: Hardware-PWM (100 Hz, LEDC) schaltet Anodenspannung
8	I2C_SDA	Bi-Dir.	I²C Daten → 4× MCP23017

9	I2C_SCL	Output	I²C Takt → 4× MCP23017
10	BTN_LIGHT	Input	Taster LIGHT (INPUT_PULLUP, aktiv LOW)
11	BTN_DOWN	Input	Taster DOWN (INPUT_PULLUP, aktiv LOW)
12	BTN_UP	Input	Taster UP (INPUT_PULLUP, aktiv LOW)
13	BTN_SET	Input	Taster SET (INPUT_PULLUP, aktiv LOW)
15	HV_TUBE_PIN_2	Output	(optional) Pro-Röhre-TLP627 Minutenzehner, nur bei HV_PER_TUBE_DIMMER
16	HV_TUBE_PIN_3	Output	(optional) Pro-Röhre-TLP627 Minuteneiner, nur bei HV_PER_TUBE_DIMMER
17	HV_TUBE_PIN_4	Output	(optional) Pro-Röhre-TLP627 Sekundenzehner, nur bei HV_PER_TUBE_DIMMER
18	HV_TUBE_PIN_5	Output	(optional) Pro-Röhre-TLP627 Sekundeneiner, nur bei HV_PER_TUBE_DIMMER
21	NEO_DATA	Output	WS2812B DIN (10 LEDs, verkettete Kette)
38	HV_TUBE_PIN_0	Output	(optional) Pro-Röhre-TLP627 Stundenzehner, nur bei HV_PER_TUBE_DIMMER
47	HV_TUBE_PIN_1	Output	(optional) Pro-Röhre-TLP627 Stundeneiner, nur bei HV_PER_TUBE_DIMMER
48	IR_RECV	Input	VS1838B demodulierter 38-kHz-Ausgang

Echtzeituhr DS1302

Der DS1302 (U2) ist eine batteriegepufferte Echtzeituhr mit ThreeWire-Interface (kein I²C/SPI-Standard). Die Kommunikation erfolgt über drei GPIOs: CE (Chip Enable, GPIO2), IO (Daten, GPIO4) und CLK (Takt, GPIO5). Der 32,768-kHz-Quarz Y1 liefert die Zeitbasis. Die CR2032-Backup-Batterie (BT1) hält die Uhrzeit auch bei Stromausfall. In der Firmware wird die Bibliothek «Rtc by Makuna» verwendet (Klasse RtcDS1302).

IR-Empfänger VS1838B

Der VS1838B (U3) ist ein integrierter IR-Empfänger mit eingebautem 38-kHz-Bandpassfilter und Demodulator. Er liefert am Ausgang (GPIO48) ein NF-Signal, das direkt von der IRremoteESP8266-Bibliothek dekodiert wird. Unterstützte Protokolle: NEC, Samsung, Sony, RC5, RC6 u.a. Alle empfangenen Codes werden gegen die im NVS gespeicherten angelernten Codes verglichen.

PCB 2 – Nixie Display Board

Das Nixie Display Board (Rev 2.01, 2026-04-06) enthält die gesamte Anzeigeelektronik. Vier MCP23017 I²C-GPIO-Expander stellen 64 digitale Ausgänge bereit, von denen 60 jeweils einen SMBTA42-NPN-Transistor ansteuern. Jeder Transistor schaltet eine Nixie-Kathode auf GND. Sechs WS2812B-SMD-LEDs (Pixel 0–5, GRB) liefern die Röhrenhintergrundbeleuchtung; vier WS2812B-THT-LEDs YF923 (Pixel 6–9, RGB) dienen als Trennpunkt-LEDs zwischen den Zifferngruppen.

Optional trägt das Board 6 zusätzliche TLP627-Optokoppler (U5–U10), je einen pro Röhre, als Ersatz für den einen gemeinsamen Anodenschalter auf dem Logic Board (siehe «Anoden-Dimmung» im vorigen Kapitel).

Schaltplan

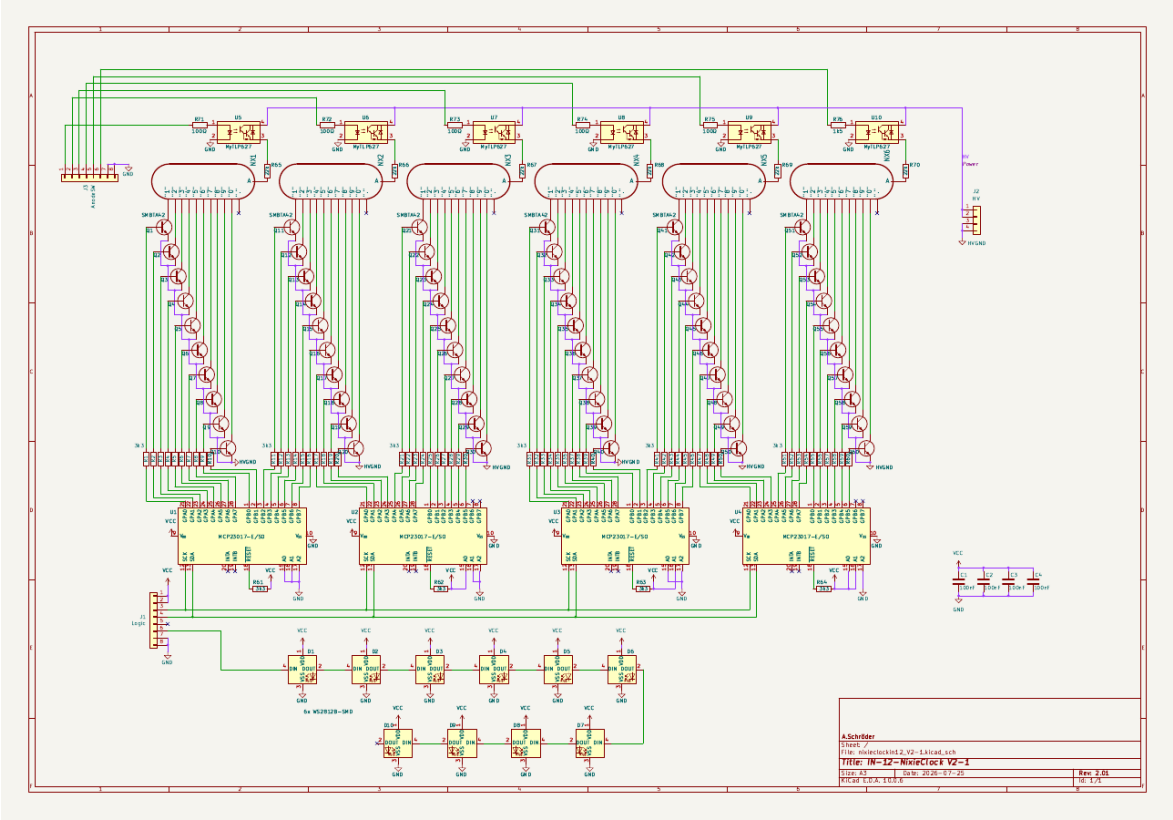


Abb. 3 – Nixie Display Board Rev 2.01 Schaltplan

PCB-Layout

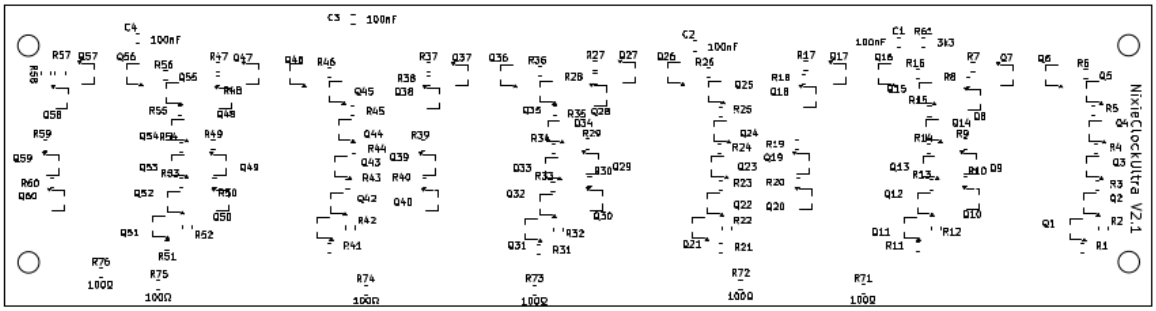


Abb. 4a – Nixie Display Board Rev 2.01 PCB-Layout Oberseite

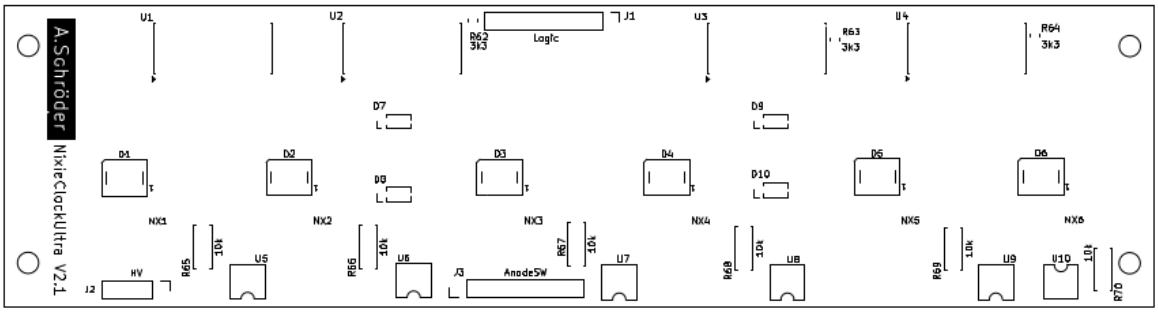


Abb. 4b – Nixie Display Board Rev 2.01 PCB-Layout Unterseite

Komponenten

Ref.	Bauteil	Anzahl	Funktion
U1–U4	MCP23017-E/SO (SOIC-28)	4	16-bit I²C GPIO-Expander, je 16 Ausgänge
Q1–Q60	SMBTA42 (TSOT-23)	60	NPN 300 V Hochvolt-Transistor als Kathodenschalter
NX1–NX6	IN-12A	6	Numerische Nixie-Röhre, Aufsicht, 10 Kathoden (0–9)
D1–D6	WS2812B-SMD	6	Röhrenhintergrundbeleuchtung, Pixel 0–5 (GRB)
D7–D10	WS2812B-THT (YF923)	4	Trennpunkt-LEDs, Pixel 6–9 (RGB-Farbreihenfolge)
R1–R64	3,3 kΩ (0805)	64	Basis-Vorwiderstände für SMBTA42
R71–R75	100 Ω (0805)	5 (opt.)	LED-Vorwiderstände Pro-Röhre-TLP627 U5–U9 (~20 mA)
R76	1,5 kΩ (0805)	1 (opt.)	LED-Vorwiderstand TLP627 U10 (Sekundeneiner, ~1,4 mA) — schnelles Abschalten, weicher Sekundenwechsel; Abweichung von R71–R75 gewollt
R65–R70	22 kΩ (THT axial)	6	Anodenwiderstände (Strombegrenzung je Röhre) — nicht kleiner wählen, sonst > 1 A an 5 V
C1–C4	100 nF (0805)	4	Abblockkondensatoren je MCP23017 (VCC gegen GND)
J1	PinHeader 1×8 (2,54 mm)	1	Inter-Board Logik-Signale ← Logic Board J3
J2	PinHeader 1×4 (2,54 mm)	1	Inter-Board HV-Versorgung ← Logic Board J4
U5–U10	TLP627 (DIP-4)	6 (optional)	Pro-Röhre-Anodenschalter — ersetzt bei Bestückung den gemeinsamen Logic-Board-Schalter U7 für unabhängiges Dimmen jeder Röhre

MCP23017 Ansteuerungskonzept

Die vier MCP23017 (U1–U4) sind am gemeinsamen I²C-Bus (SDA=GPIO8, SCL=GPIO9) angeschlossen; die Pull-ups R1 (SCL) und R3 (SDA), je 3,3 kΩ nach +3,3 V, sitzen auf dem Logic Board. Der Bus läuft mit 100 kHz; bei 400 kHz und 1 MHz scheiterte im Test jeder Schreibvorgang (Ursache ungeklärt, Verdacht: die MCP23017 laufen an 5 V und erwarten einen High-Pegel von mindestens 4 V, die Pull-ups ziehen nur auf 3,3 V). Versorgt werden die MCP23017 mit 5 V über J3. Die I²C-Adressen werden durch die Hardware-Adresspins A0, A1, A2 festgelegt:

IC	I²C-Adresse	A2	A1	A0	Zuständige Röhre(n)
U1	0x20	0	0	0	Tube 0 (Stundenzehner), Tube 1 Bits 10–15

U2	0x21	0	0	1	Tube 1 Bits 0–3, Tube 2 (Minutenzehner)
U3	0x22	0	1	0	Tube 3 (Minuteneiner), Tube 4 Bits 10–15
U4	0x23	0	1	1	Tube 4 Bits 0–3, Tube 5 (Sekundeneiner)

Jeder MCP23017 verfügt über zwei 8-Bit-Ports (GPA0–7, GPB0–7), also 16 Ausgänge pro IC. Alle 64 Ausgänge werden beim Start im `nixieInit()`-Aufruf als Output konfiguriert (IODIR-Register = 0x00). Die Ansteuerung erfolgt über das OLATA-Register (0x14) mit einem einzigen I²C-Schreibzugriff, der beide Ports gleichzeitig setzt (16 Bits, Low-Byte GPA, High-Byte GPB).

Tube-zu-MCP-Bit-Zuordnung

Die Verteilung der 60 Kathoden (6 Röhren × 10 Ziffern) auf die MCP-Bits ist in der Firmware als Lookup-Tabelle `digitPin[6][10]` implementiert (Struktur `DigitPin {uint8_t mcp; uint8_t bit;}`):

Tube	Anzeige	MCP	GPA/GPB-Bits
0	Stundenzehner	0x20	Bits 0–9 (Ziffern 1,2,3,4,5,6,7,8,9,0)
1	Stundeneiner	0x20+0x21	MCP0 Bits 10–15 + MCP1 Bits 0–3
2	Minutenzehner	0x21	Bits 4–13
3	Minuteneiner	0x22	Bits 0–9
4	Sekundenzehner	0x22+0x23	MCP2 Bits 10–15 + MCP3 Bits 0–3
5	Sekundeneiner	0x23	Bits 4–13

Ein Digit-Wert von 10 bedeutet «Röhre blank» — es wird kein Bit gesetzt, alle Kathoden bleiben HIGH, die Röhre zeigt nichts an.

Transistor-Kathodenschaltung

Jede Nixie-Kathode wird über einen SMBTA42-NPN-Transistor auf GND gezogen. Der SMBTA42 ist für 300 V Kollektor-Emitter-Spannung ausgelegt — notwendig, da die Nixie-Anoden auf ~170 V liegen. Der Basis-Vorwiderstand beträgt 3,3 kΩ. Bei einem GPIO-High von 3,3 V fließt ein Basisstrom von ca. 1 mA, was den Transistor sicher in die Sättigung treibt (typischer Verstärkungsfaktor hFE ≥ 100). Der Kollektor-Strom (Katodenleuchtstrom) beträgt typisch 1–3 mA, begrenzt durch den internen Widerstand der Nixie-Röhre.

WS2812B NeoPixel-Konfiguration

Die 10 WS2812B-LEDs sind als verkettete Kette an GPIO21 angeschlossen. SMD- und THT-Varianten haben unterschiedliche Farbreihenfolgen, was in der Firmware berücksichtigt wird:

Pixel	Typ	Farbreihenfolge	Position / Funktion

0–5	WS2812B-SMD	GRB	Röhrenhintergrundbeleuchtung (6 LEDs)
6–9	WS2812B-THT YF923	RGB	Trennpunkt-LEDs zwischen Zifferngruppen (4 LEDs)

Firmware-Architektur

Die Firmware ist als Arduino-Sketch für den ESP32-S3 implementiert und in mehrere `.ino`-Module aufgeteilt. Arduino kompiliert alle `.ino`-Dateien eines Verzeichnisses automatisch zu einer einzigen Compilation Unit — globale Variablen und Funktionen sind dateienübergreifend sichtbar.

Modulübersicht

Datei	Zeilen	Inhalt / Verantwortlichkeit
NixieClockUltra.ino	528	Globals, setup(), loop(), Edit-Mode-FSM (Zeit+Datum), Nacht-Modus-Globals, Röhrentest-Globals
nixie_driver.ino	91	nixieInit(), nixieWrite(), MCP23017-I²C-Abstraktion, Shadow-Register, Mutex
display.ino	91	setDisplayTime(), setDisplayDate(), commitDigits() (weicher Ziffernwechsel, Bitmaske via computeChangedMask()), Slot-Animation
digit_fade.ino	100	startDigitFade(), updateDigitFade(), cancelDigitFade() — non-blocking Crossfade über HV-Dimmer-Duty, pro Röhre per Bitmaske
buttons.ino	127	Entprell-FSM für 4 Taster, Kurz-/Langdruck, Edit-Mode Zeit+Datum
rtc.ino	18	readRTC(), writeRTC() via DS1302/ThreeWire, liest auch Tag/Monat/Jahr
night_mode.ino	34	LDR-Abtastung (GPIO6, ADC1), updateNightMode(), Zeitbereich-Logik
hv_dimmer.ino	94	hvDimmerInit(), hvDimmerSoftStart(), hvDimmerSetDutyAll(), hvDimmerSetDutyTube(), hvDimmerSetDutyTube10() — LEDC-Hardware-PWM für TLP627 auf Anodenspannung; bei HV_PER_TUBE_DIMMER 6 unabhängige Kanäle statt einem gemeinsamen
neo_animation.ino	107	Rainbow, Statisch, Puls, Slot, Nacht-Modus-Dimming, Datumsanzeige-Override
ir_remote.ino	107	executeAction(), dispatchIRAction(), handleIR(), 8 IR-Aktionen
tube_test.ino	50	startTubeTest(), updateTubeTest(), stopTubeTest() — non-blocking Röhrentest-State-Machine
web_server.ino	787	Eingebettetes HTML/JS, alle API-Handler, WiFi-Setup (setupWifiRadio() + setupWifi()), NTP, mDNS
boot_diag.ino	128	Dauerhaftes Startprotokoll im NVS: Reset-Grund, letzter Start-Schritt und Laufzeit der letzten 20 Starts

Reine Interpolationsmathematik für den Crossfade liegt zusätzlich in digit_fade_math.h (header-only, host-testbar ohne Arduino-Framework, siehe test/digit_fade_math_test.cpp). Analog dazu liegt die reine Ziffern-/Testende-Logik des Röhrentests in tube_test_math.h (siehe test/tube_test_math_test.cpp).

Bibliotheksabhängigkeiten

Bibliothek	Autor	Version	Funktion
Adafruit NeoPixel	Adafruit	aktuell	WS2812B-Ansteuerung (GRB + RGB)

Rtc by Makuna	Michael C. Miller	aktuell	DS1302 RTC über ThreeWire-Protokoll
AsyncTCP	me-no-dev	aktuell	Async-TCP-Stack für ESP32
ESPAsyncWebServer	me-no-dev	aktuell	Nicht-blockierender HTTP-Server
ArduinoJson	Benoit Blanchon	v6.x	JSON-Serialisierung (v7 inkompatibel!)
IRremoteESP8266	David Conran	aktuell	IR-Empfang NEC/Samsung/Sony/RC5/RC6 …

⚠️ ArduinoJson muss in Version 6.x installiert sein — Version 7 hat eine inkompatible API.

Setup-Ablauf

Die setup()-Funktion wird einmalig nach dem Einschalten ausgeführt:

Schritt	Funktion / Aktion	Details
1	Serial.begin(115200), bootDiagInit()	Debug-Ausgabe aktivieren, Startprotokoll ausgeben und neuen Eintrag anlegen
2	pinMode(BTN_*, INPUT_PULLUP)	Alle 4 Taster-GPIOs konfigurieren (IO10–IO13)
3	strip.begin() / clear()	NeoPixel initialisieren, alle LEDs aus
4	hvDimmerInit()	LEDC-PWM anlegen, Duty zunächst 0 (Anoden zu, Einschaltlast entzerren)
5	prefs.begin("nixie")	NVS öffnen: Helligkeit, Anim, SlotIval, IR-Codes, WiFi, Nacht-Modus-Konfiguration inkl. hvDimPct, sowie softFadeSecondEnabled/softFadeDateEnabled/slotSpeedPct laden
6	nixieInit()	I²C starten, alle 4 MCP23017 auf Output, alle Bits 0
7	Rtc.Begin() + readRTC()	DS1302 starten, Uhrzeit + Datum laden, Ziffern auf Kathoden schreiben (Anoden noch zu)
8	setupWifiRadio()	Funkmodul + AP starten (HF-Kalibrierung = Stromspitze, vor den Röhren)
9	hvDimmerSoftStart()	Röhren nacheinander aufblenden (6 × 200 ms)
10	setupWifi()	Heimnetz verbinden (max. 20 s) + NTP; mDNS
11	setupWebServer()	Alle API-Routen registrieren, server.begin()
12	irrecv.enableIRIn()	IR-Empfänger aktivieren (GPIO48)
13	startFadeIn = true	Fade-In-Flag setzen → NeoPixel blenden in loop() ein

Globale State-Variablen

Variable	Typ	Beschreibung
displayDigits[6]	uint8_t[6]	Aktuell angezeigte Ziffern (0–9; 10 = blank)
brightLevel	uint8_t	Helligkeitsstufe 0–3 (Index in BRIGHTNESS_LEVELS[])
neoBright	uint8_t	NeoPixel-Feinwert 10–255 (Hintergrund Pixel 0–5)

colonBright	uint8_t	NeoPixel-Feinwert (Trennpunkte Pixel 6–9)
neoHue	uint8_t	Auto-inkrement für Rainbow-Farbverlauf
animMode	AnimMode	Aktueller Animationsmodus (Enum, s. u.)
editState	EditState	Aktueller Einstellschritt (NONE/HOUR/MIN/SEC/DAY/MONTH/YEAR)
colonAlwaysOn	bool	Trennpunkte dauerhaft an (kein Blinken)
colonStatic	bool	Trennpunkte statisch warmweiß
curDay/Month/Year	uint8_t	Lokal gecachtes Datum (Tag 1–31, Monat 1–12, Jahr 0–99)
dateShowActive	bool	Datumsanzeige nach Slot-Animation gerade aktiv
dateShowStart	uint32_t	millis()-Zeitstempel beim Start der Datumsanzeige
nightState	NightState	Aktueller Nacht-Modus-Zustand (NORMAL/DIM/DARK)
nightTimeEnabled	bool	Zeitbereich-Nacht-Modus aktiv
nightStart/End	uint8_t	Start-/Endstunde des Nacht-Bereichs (0–23)
nightTimeMode	uint8_t	0 = Gedimmt, 1 = Dunkel
ldrEnabled	bool	Lichtsensor-Steuerung aktiv
ldrThreshold	uint16_t	ADC-Schwellwert (0–4095); <= Schwellwert → Dimmen
ldrReading	uint16_t	Aktueller ADC-Messwert (alle 500 ms aktualisiert)
hvDimPct	uint8_t	Röhren-Dimm-Helligkeit im Nacht-Modus in % (2–60, Default 25)
irCodes[8]	uint64_t[8]	Angelernte IR-Codes, Index = IrAction-Enum (0–7)
wifiStaConnected	bool	STA-Verbindung zum Heimnetz aktiv
ntpSynced	bool	NTP-Synchronisierung erfolgreich
slotActive	bool	Slot-Machine-Animation läuft
startFadeIn	bool	Einblend-Sequenz beim Start aktiv
softFadeSecondEnabled	bool	Weicher Ziffernwechsel im Sekundentakt aktiv
softFadeDateEnabled	bool	Weicher Ziffernwechsel bei Zeit/Datum-Übergang aktiv
slotSpeedPct	uint8_t	Slot-Machine-Rollgeschwindigkeit 20–100 % (100 = Standard)
tubeTestActive	bool	Röhrentest läuft gerade
tubeTestDigit	uint8_t	Aktuell angezeigte Testziffer (0–9)
tubeTestStepStart	uint32_t	millis() beim Anzeigen der aktuellen Testziffer

Enumerationen

```
enum AnimMode { ANIM_RAINBOW, ANIM_STATIC, ANIM_PULSE, ANIM_COUNT };

enum EditState { EDIT_NONE,
EDIT_HOUR, EDIT_MIN, EDIT_SEC,
EDIT_DAY, EDIT_MONTH, EDIT_YEAR };

enum NightState { NIGHT_NORMAL, NIGHT_DIM, NIGHT_DARK };

enum IrAction {
IR_LEARN_NONE = -1,
IR_ACTION_SET = 0, IR_ACTION_UP = 1,
```

```
IR_ACTION_DOWN = 2, IR_ACTION_BRIGHTNESS = 3,
IR_ACTION_ANIM_NEXT = 4, IR_ACTION_SLOT = 5,
IR_ACTION_COLON_TOGGLE = 6, IR_ACTION_DATE = 7,
IR_ACTION_COUNT = 8
};
```

hv_dimmer.ino – Hardware-PWM-Dimmung der Anodenspannung

Die Röhren-Dimmung im Nacht-Modus erfolgt über einen oder sechs TLP627-Optokoppler, die die Anodenspannung selbst per LEDC-Hardware-PWM (100 Hz, 10 Bit) schalten — nicht über eine Software-PWM auf den Kathoden. Der HV_PER_TUBE_DIMMER-Compile-Switch entscheidet, welche der beiden Varianten kompiliert wird:

> ⚠️ Warum 100 Hz: Der TLP627 (Darlington) schaltet deutlich langsamer ab als ein, beim Pro-Röhre-Dimmer besonders stark (nur ~1,5 mA Röhrenstrom bei ~20 mA LED-Strom → tief übersteuert). Bei 200 Hz wurden die Röhren zwischen 60 % und 2 % kaum dunkler; bei 100 Hz wirkt die Verzögerung nur halb so stark. Für 100 Hz läuft die PWM mit 10 Bit (der Core taktet LEDC mit 40 MHz, mit 8 Bit erst ab ~153 Hz möglich); hvDuty() skaliert die Werte 0–255 auf 0–1023.

Der größere Hebel ist der LED-Vorwiderstand: Bei der Sekundeneiner-Röhre ist R76 auf 1,5 kΩ erhöht (LED-Strom ~1,4 mA statt ~20 mA). Der Koppler schaltet die Röhre weiterhin voll durch, ist aber nicht mehr übersteuert und schaltet schnell ab — getestet: volle Maximalhelligkeit, deutlich weicherer Ziffernwechsel.

> ⚠️ R71–R75 bleiben bewusst bei 100 Ω — die unterschiedliche Bestückung ist gewollt: Wechseln mehrere Röhren gleichzeitig (bei 0, 10, 20 … Sekunden), blenden die Röhren mit 100 Ω schneller ganz ab als die Sekundeneiner, dann wechselt die Sekundeneiner weich, und die übrigen schießen anschließend auf ihre Zielhelligkeit — ein gestaffelter Effekt, der sehr gut wirkt. Wer alle sechs auf 1,5 kΩ setzt, verliert ihn. Bekannte Nebenwirkung: Im gedimmten Nacht-Modus kann die Sekundeneiner bei kleinen Dimmwerten etwas dunkler wirken.

```
#ifdef HV_PER_TUBE_DIMMER
// 6 unabhängige Kanäle, einer pro Röhre
void hvDimmerInit() { /* ledcAttach() je HV_TUBE_PIN_0..5, Duty 0 */ }
void hvDimmerSoftStart() { /* Röhre für Röhre 0 → 255 in HV_SOFTSTART_TUBE_MS */ }
void hvDimmerSetDutyAll(uint8_t duty0to255) { /* alle 6 Kanäle */ }
void hvDimmerSetDutyTube(uint8_t tube, uint8_t duty0to255) { /* nur dieser Kanal */
}

#else
// Ein gemeinsamer Kanal (heutige Standard-Hardware, U7)
void hvDimmerInit() {
ledcAttach(HV_SWITCH_PIN, HV_PWM_FREQ_HZ, HV_PWM_RES_BITS);
ledcWrite(HV_SWITCH_PIN, 0); // Anoden zu – Aufblenden per hvDimmerSoftStart()
}
void hvDimmerSoftStart() { /* alle gemeinsam 0 → 255 in 6 × HV_SOFTSTART_TUBE_MS */
}
void hvDimmerSetDutyAll(uint8_t duty0to255) { ledcWrite(HV_SWITCH_PIN, duty0to255);
}
// Röhrenindex wird ignoriert – es gibt nur den einen gemeinsamen Schalter.
void hvDimmerSetDutyTube(uint8_t /*tube*/, uint8_t duty0to255)
{ ledcWrite(HV_SWITCH_PIN, duty0to255); }
#endif
```

hvDimmerSetDutyAll() wird bei einem Wechsel von nightState in loop() aufgerufen (Guard nightState !=

prevNightState), nicht bei jedem Durchlauf — setzt alle 6 Röhren gemeinsam auf dieselbe Ziel-Helligkeit, unabhängig davon ob HV_PER_TUBE_DIMMER aktiv ist: NIGHT_NORMAL → Duty 255, NIGHT_DIM → Duty hvDimPct*255/100 (2–60 %), NIGHT_DARK → Duty 0. hvDimmerSetDutyTube() wird ausschließlich aus der Fade-State-Machine in digit_fade.ino heraus aufgerufen — kein zusätzlicher CPU-Overhead im Normalbetrieb. Ein separater Blitzschutz für Sekundenwechsel ist nicht nötig, da die Kathoden-Ansteuerung unabhängig von der Anodendimmung läuft.

digit_fade.ino – Weicher Ziffernwechsel

commitDigits() in display.ino ist die zentrale Stelle, über die alle Ziffernänderungen laufen (setDisplayTime(), setDisplayDate() sowie die Soft-Varianten setDisplayTimeSoft()/setDisplayDateSoft()). Sie berechnet per computeChangedMask() (digit_fade_math.h) eine Bitmaske der Röhren, deren Ziffer sich ändert (Bit i = Tube-Index i, 0=HZ…5=SE) — ist die Maske 0, ändert sich nichts und der Aufruf wird ignoriert. Sonst: fadeMs == 0 oder nightState == NIGHT_DARK → sofortiger Hart-Wechsel über nixieWrite() (in NIGHT_DARK ist die Anodenspannung ohnehin aus, ein Fade wäre unsichtbar); fadeMs > 0 und nightState != NIGHT_DARK (also auch in NIGHT_DIM) → startDigitFade() mit der berechneten Maske als Parameter.

startDigitFade()/updateDigitFade() bilden eine non-blocking State-Machine (angetrieben aus loop()), die den HV-Dimmer-Duty nur für die Röhren in der Fade-Maske (hvDimmerSetDutyTube10(), 10 Bit) in 5-ms-Schritten (DIGIT_FADE_STEP_MS) erst auf einen Minimalwert abblendet, bei Minimalhelligkeit die Zielziffern schreibt (nixieWrite()), und wieder auf die Ziel-Helligkeit (fadeMaxDuty) aufblendet. fadeMaxDuty wird beim Fade-Start einmalig ermittelt: 1023 bei NIGHT_NORMAL, hvDuty(hvDimPct*255/100) bei NIGHT_DIM — der Fade wirkt im Dimm-Modus also konsistent gedimmt statt kurz auf 100 % aufzublitzen. Der Minimalwert (fadeMinDuty) wird proportional zu fadeMaxDuty berechnet, nicht als fixer Wert — das vermeidet einen Integer-Underflow bei sehr niedriger Dimm-Helligkeit (siehe Kapitel «Entwicklungsgeschichte» auf der Website). Röhren außerhalb der Maske werden während des gesamten Fades nicht angefasst — mit aktiviertem HV_PER_TUBE_DIMMER bleiben sie sichtbar unverändert hell; ohne die Hardware-Option dimmt hvDimmerSetDutyTube() ohnehin den einen gemeinsamen Schalter, sodass sich am heutigen Erscheinungsbild (alle 6 dimmen gemeinsam) nichts ändert.

Helligkeitskurve: Der Duty folgt keiner linearen Rampe, sondern einer Potenzkurve (fadeDutyCurve(), Exponent DIGIT_FADE_GAMMA = 4,0, an der Hardware abgestimmt): Abblenden max·(1−t)^γ, Aufblenden max·t^γ. Nixies werden erst bei kleinem Tastverhältnis sichtbar dunkler (logarithmische Wahrnehmung, Abschaltverzögerung der TLP627) — eine lineare Rampe wirkte wie ein kurzes Wegtauchen am Ende. Die Kurve verbringt den Großteil der Zeit bei kleinen Tastgraden; γ = 1 entspräche der linearen Rampe, größere Werte verlängern die dunkle Phase. Gerechnet wird in 10 Bit, damit es am dunklen Ende keine sichtbaren Stufen gibt.

> ⚠️ Grenze: Wegen der Abschaltverzögerung der TLP627 hat jeder Tastgrad > 0 eine Mindesthelligkeit (wirksam ≈ Tastgrad + Verzögerung/Periode). Die Kurve blendet weich bis dorthin, der letzte Schritt auf 0 bleibt ein kleiner Sprung — ein höherer Exponent ändert daran nichts. Abhilfe: schnelleres Abschalten der Koppler — bei der Sekundeneiner ist dafür R76 auf 1,5 kΩ erhöht, der Sprung ist dort praktisch verschwunden; bei den übrigen Röhren (100 Ω) fällt er wegen der seltenen Wechsel nicht auf.

Die Dauer wird über fadeMs gesteuert (je zur Hälfte Ab-/Aufblenden): DIGIT_FADE_MS = 900 ms für Sekundentakt und Zeit/Datum-Wechsel. Damit ein Wechsel fertig ist, bevor die nächste Sekunde kommt, liest

loop() die RTC alle RTC_POLL_MS = 50 ms (vorher 500 ms — der Sekundenwechsel wurde um bis zu eine halbe Sekunde verspätet erkannt, der Takt schwankte sichtbar). Die Kurvenmathematik (fadeDutyCurve(), lineare fadeDutyForStep() für den Sanftanlauf) und die Masken-Berechnung (computeChangedMask()) liegen in digit_fade_math.h und sind per Host-Unit-Test ohne Arduino-Framework testbar.

cancelDigitFade() schließt einen laufenden Fade sofort ab: Zielziffern schreiben und die betroffenen Röhren (Fade-Maske) auf fadeMaxDuty zurücksetzen — und wird vor startSlotAnimation() sowie beim Eintritt in den Edit-Modus aufgerufen, als Absicherung gegen eine Race zwischen laufendem Fade und neuer Display-Aktivität.

Slot-Machine-Geschwindigkeit

slotSpeedPct (20–100 %, Standard 100) skaliert sowohl slotRollIntervalMs als auch die gestaffelten Stopp-Zeitpunkte je Röhre (slotStopMs[i]) in startSlotAnimation() (display.ino) — bei kleineren Werten rollen die Ziffern langsamer und die Animation dauert insgesamt länger.

Nacht-Modus (night_mode.ino)

updateNightMode() wird jeden Loop-Durchlauf aufgerufen und bestimmt den nightState in drei Stufen:

Priorität	Bedingung	Ergebnis
1 (höchste)	nightTimeEnabled && Stunde im konfigurierten Bereich	NIGHT_DIM oder NIGHT_DARK (je nach nightTimeMode)
2	ldrEnabled && ldrReading <= ldrThreshold	NIGHT_DIM (LDR kann nur dimmen, nicht dunkel)
3 (Fallback)	keine der obigen Bedingungen	NIGHT_NORMAL

Der LDR (GPIO6, ADC1) wird immer alle 500 ms abgetastet — unabhängig davon ob ldrEnabled gesetzt ist. So ist der Rohwert im Web-UI immer aktuell. LDR→VCC, 100 kΩ→GND: Helligkeit = hoher ADC-Wert, Dunkelheit = niedriger Wert.

Datumsanzeige

Nach jeder Slot-Animation und bei IR-Taste DATUM wird dateShowActive=true gesetzt. Für 5000 ms (DATE_SHOW_MS) zeigen die Röhren das Datum im Format TT MM JJ. Die NeoPixel-Ausgabe in neo_animation.ino überschreibt dabei alle Pixel: Hintergrund (0–5) und obere Trennpunkte (Pixel 6, 8) werden gelöscht. Nur die unteren Trennpunkte (Pixel 7 und 9) leuchten in warmweiß — ein visueller Hinweis, dass Datum statt Uhrzeit angezeigt wird.

Röhrentest (tube_test.ino)

Diagnose-Feature: Auf Anfrage zeigen alle 6 Röhren synchron dieselbe Ziffer, in Schritten von 0 bis 9, je TUBE_TEST_STEP_MS (2000 ms) — insgesamt ~20 s. Zweck: jede Ziffer auf jeder Röhre einmal sichtbar durchprüfen (defekte Kathode, kalter Lötpunkt an Transistor/MCP23017), was die Slot-Machine-Animation nicht

leistet (dort steht nie jede Röhre synchron auf derselben Ziffer still).

startTubeTest() bricht konkurrierende Display-Nutzer ab (cancelDigitFade(), slotActive=false, dateShowActive=false, editState=EDIT_NONE), erzwingt hvDimmerSetDutyAll(255) unabhängig vom Nacht-Modus und schreibt Ziffer 0 sofort hart auf alle Röhren. Erneuter Aufruf während eines laufenden Tests setzt ihn einfach auf Ziffer 0 zurück (kein Fehlerfall). updateTubeTest() (aus loop()) schaltet alle TUBE_TEST_STEP_MS eine Ziffer weiter, bis tubeTestIsFinished() (aus tube_test_math.h) nach Ziffer 9 true liefert → stopTubeTest(). stopTubeTest() stellt die HV-Duty passend zum aktuellen nightState wieder her (identischer Switch wie im Nacht-Modus-Block in loop()), synchronisiert prevNightState (verhindert eine doppelte Duty-Anwendung im nächsten loop()-Durchlauf) und zeigt sofort wieder die aktuelle Uhrzeit an. Aufruf ohne laufenden Test ist ein No-op.

Guards in loop(): Der bestehende Nacht-Modus-Duty-Block (nightState != prevNightState) prüft zusätzlich ! tubeTestActive, damit ein Nacht-Modus-Wechsel die erzwungene volle Helligkeit während des Tests nicht überschreibt. Ebenso pausiert handleEditMode() sowie der Sekundentakt-/Slot-Trigger-Block vollständig, solange tubeTestActive gesetzt ist.

> ⚠️ Bekannte Einschränkung: Die IR-Fernbedienung (handleIR()) läuft unguardet vor diesen Prüfungen — SLOT/DATE/SET per Fernbedienung während eines laufenden Tests können die Anzeige durcheinanderbringen bzw. den Editiermodus nach Testende hängen lassen. Bewusst akzeptierter Edge-Case (seltene gleichzeitige IR-Eingabe während der ~20 s Testdauer), nicht behoben.

Web-API: POST /api/tubetest/start und POST /api/tubetest/stop rufen direkt startTubeTest()/stopTubeTest() aus dem Async-Webserver-Task auf — analog zu /api/settime, nicht zum Flag-only-Muster von /api/slot. nixieWrite() ist über den bestehenden FreeRTOS-Mutex gegen den gleichzeitigen Zugriff aus loop() abgesichert.

nixie_driver.ino – Direkte MCP23017-Ansteuerung

Der Nixie-Treiber implementiert die direkte, multiplexingfreie Ansteuerung der 6 Röhren. Kernfunktionen:

nixieInit()

Initialisiert den I²C-Bus (Wire.begin(SDA=8, SCL=9)) und konfiguriert alle vier MCP23017 durch Schreiben von 0x00 in die IODIRA- (0x00) und IODIRB- (0x01) Register — alle 16 Pins werden zu Ausgängen. Anschließend werden alle Ausgänge auf LOW (0) gesetzt und ein FreeRTOS-Mutex (nixieMutex) für thread-sichere Zugriffe erstellt.

nixieWrite(uint8_t digits[6])

Berechnet für jede der 6 Röhren das zugehörige MCP-Bit via `digitPin[tube][digit]`-Lookup (O(1)), baut den neuen Zustand in `newState[4]` auf und schreibt nur die MCPs, deren Shadow-Register (`mcpState[]`) sich geändert haben — minimiert I²C-Traffic. Der gesamte Zugriff ist durch den FreeRTOS-Mutex geschützt, da nixieWrite() sowohl aus dem Loop-Task als auch aus dem asynchronen Web-Server-Task aufgerufen werden kann.

```
// Einziger I²C-Schreibzugriff pro geändertem MCP (beide Ports gleichzeitig):
Wire.beginTransmission(addr);
Wire.write(MCP_OLATA); // Register 0x14
```

```
Wire.write((uint8_t)(val & 0xFF)); // GPA (Low-Byte)
Wire.write((uint8_t)((val >> 8)&0xFF));// GPB (High-Byte)
Wire.endTransmission();
```

Web-API

Der HTTP-Server läuft auf Port 80, erreichbar über 192.168.4.1 (AP) oder http://nixieclockcs.local (Heimnetz via mDNS). POST-Endpunkte erwarten JSON (Content-Type: application/json).

Pfad	Methode	Request-Body / Antwort
/	GET	Vollständiges Web-Interface (HTML/CSS/JS eingebettet als PROGMEM)
/api/status	GET	{time, date, neoBright, animMode, slotIval, slotSpeed, colonOn, colonStatic, colonBright, slot, tubeTest, tubeTestDigit, nightState, ldrVal, wifiSta, ntpSynced}
/api/settime	POST	{"h":H,"m":M,"s":S} + optional {"d":D,"mo":Mo,"y":Y} → RTC schreiben
/api/neobright	POST	{"val":10..255} → NeoPixel-Feinwert + NVS
/api/anim	POST	{"mode":0..2} → Animationsmodus + NVS
/api/slotinterval	POST	{"interval":0..4} → Slot-Intervall (SlotInterval-Enum) + NVS
/api/colonbright	POST	{"val":1..100} → Trennpunkt-Helligkeit + NVS
/api/colonon	POST	{"enabled":true|false} → Dauerhaft an/aus + NVS
/api/colonstatic	POST	{"enabled":true|false} → Statisch warmweiß + NVS
/api/slot	POST	(kein Body) → Slot-Machine-Animation sofort starten
/api/slotspeed	POST	{"val":20..100} → Slot-Machine-Rollgeschwindigkeit + NVS
/api/tubetest/start	POST	(kein Body) → Röhrentest starten (bzw. auf Ziffer 0 zurücksetzen, falls bereits aktiv)
/api/tubetest/stop	POST	(kein Body) → Röhrentest sofort beenden (No-op, falls nicht aktiv)
/api/softfade	GET	{sec, date} — weicher Ziffernwechsel Sekundentakt/Datum-Übergang
/api/softfade	POST	{"sec":true|false,"date":true|false} → weicher Ziffernwechsel + NVS
/api/nightmode	GET	{ntEn, ntFrom, ntTo, ntMode, hvDimPct, ldrEn, ldrThr, ldrVal, state}
/api/nightmode	POST	{ntEn, ntFrom, ntTo, ntMode, hvDimPct, ldrEn, ldrThr} → Nacht-Modus + NVS
/api/wifi	GET	{mode, staSsid, staIp, ntp}
/api/wifi	POST	{"ssid":"…","pass":"…"} → STA verbinden, Neustart
/api/ir/status	GET	8 IR-Code-Einträge (je action + code)
/api/ir/learn	POST	{"action":"SET"|"UP"|"DOWN"|"BRIGHTNESS"|"ANIM_NEXT"|"SLOT"|"COLON_TOGGLE"|"DATUM"}
/api/ir/clear	POST	{"action":"…"} → IR-Code löschen + NVS

NVS-Persistenz

Alle Einstellungen werden über die Arduino-`Preferences`-Bibliothek im ESP32-NVS-Flash gespeichert (Namespace "nixie"). Beim Start werden sie automatisch geladen. Das Startprotokoll liegt getrennt davon im Namespace "bootlog" (Schlüssel "log", Blob).

NVS-Schlüssel	Typ	Default	Inhalt
bright	UInt8	3	Helligkeitsstufe (0–3)
neoBright	UInt8	30	NeoPixel-Feinwert Hintergrund
colonBright	UInt8	80	NeoPixel-Feinwert Trennpunkte
animMode	UInt8	0	Animationsmodus (0=Rainbow, 1=Statisch, 2=Puls)
slotIval	UInt8	0	Slot-Intervall (SlotInterval-Enum: 0=Aus, 1=10s, 2=1min, 3=15min, 4=1h)
slotSpeed	UInt8	100	Slot-Machine-Rollgeschwindigkeit (20–100 %)
sfSecEn	Bool	false	Weicher Ziffernwechsel Sekundentakt aktiv
sfDateEn	Bool	false	Weicher Ziffernwechsel Zeit/Datum aktiv
colonOn	Bool	false	Trennpunkte dauerhaft an
colonStatic	Bool	false	Trennpunkte statisch warmweiß
ntEn	Bool	false	Nacht-Modus Zeitbereich aktiv
ntFrom	UInt8	23	Nacht-Modus Startzeit (Stunde 0–23)
ntTo	UInt8	7	Nacht-Modus Endzeit (Stunde 0–23)
ntMode	UInt8	0	Nacht-Modus Typ (0=Gedimmt, 1=Dunkel)
hvDimPct	UInt8	25	Röhren-Dimm-Helligkeit (2–60 %)
ldrEn	Bool	false	Lichtsensor-Steuerung aktiv
ldrThr	UInt16	512	LDR-Schwellwert ADC (0–4095)
wifiSsid	String	""	Heimnetz SSID
wifiPass	String	""	Heimnetz Passwort
ir_SET	UInt64	0	IR-Code für IR_ACTION_SET
ir_UP	UInt64	0	IR-Code für IR_ACTION_UP
ir_DOWN	UInt64	0	IR-Code für IR_ACTION_DOWN
ir_BRIGHTNESS	UInt64	0	IR-Code für IR_ACTION_BRIGHTNESS
ir_ANIM_NEXT	UInt64	0	IR-Code für IR_ACTION_ANIM_NEXT
ir_SLOT	UInt64	0	IR-Code für IR_ACTION_SLOT
ir_COLTOGGLE	UInt64	0	IR-Code für IR_ACTION_COLON_TOGGLE
ir_DATE	UInt64	0	IR-Code für IR_ACTION_DATE (Datumsanzeige)

Entwickler-Workflow

Arduino IDE einrichten

Voraussetzung: Arduino IDE 2.x. ESP32-Boardunterstützung installieren:

```
Datei → Einstellungen → Zusätzliche Boardverwalter-URLs:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/
package_esp32_index.json
```

Dann: Werkzeuge → Board → Boardverwalter → «esp32» suchen → Paket von Espressif Systems installieren (Version 3.x). Board auswählen: ESP32S3 Dev Module.

Bibliotheken installieren

Alle Bibliotheken über den Arduino Library Manager installieren (Werkzeuge → Bibliotheken verwalten):

Bibliothek	Suchbegriff im Library Manager
Adafruit NeoPixel	Adafruit NeoPixel
Rtc by Makuna	Rtc by Makuna
AsyncTCP	AsyncTCP (me-no-dev)
ESPAsyncWebServer	ESPAsyncWebServer (me-no-dev)
ArduinoJson	ArduinoJson (v6.x!)
IRremoteESP8266	IRremoteESP8266

⚠️ ArduinoJson: Explizit Version 6.x wählen — Version 7 ist API-inkompatibel.

ESP32-S3 USB-Einstellungen — kritisch!

Das Logic Board verwendet die native USB-Schnittstelle des ESP32-S3 (USB-OTG, GPIO19/20) — es gibt keinen separaten USB-UART-Chip. Der ROM-Bootloader des ESP32-S3 meldet sich ab Werk als USB-CDC-Port, sodass das erste Flashen problemlos funktioniert.

Die Gefahr liegt im zweiten Schritt: Wird die Firmware mit der Einstellung «USB CDC On Boot: Disabled» kompiliert und geflasht, schaltet die laufende Firmware das CDC ab. Beim nächsten Start erscheint kein COM-Port mehr — die Arduino IDE kann keinen automatischen Reset in den Bootloader-Modus auslösen. Da der BOOT-Taster SW5 nicht bestückt ist, gibt es keinen manuellen Ausweg: der ESP32-S3 ist danach über USB nicht mehr programmierbar.

⚠️ SW5 (BOOT) ist nicht bestückt. Einmal mit «USB CDC On Boot: Disabled» geflasht bedeutet: nie wieder über USB flashbar — bis GPIO0 über einen Lötpunkt manuell auf GND gezogen wird.

Folgende Einstellungen müssen unter Werkzeuge vor jedem Kompilieren und Hochladen gesetzt sein:

Einstellung	Wert	Bemerkung
Board	ESP32S3 Dev Module	Espressif ESP32-S3-Paket v3.x
USB Mode	Hardware CDC and JTAG	Natives USB des ESP32-S3 aktivieren
Upload Mode	UART0 / Hardware CDC	Upload über dieselbe USB-Buchse
USB CDC On Boot	Enabled	KRITISCH: Disabled sperrt USB-Zugang dauerhaft (SW5 nicht bestückt)
USB DFU On Boot	Disabled	Nicht benötigt
Flash Size	4 MB (32 Mb)	Passend zum ESP32-S3-WROOM-1
Upload Speed	921600	

Firmware flashen

1. Micro-USB-Kabel am Logic Board J2 anschließen (5 V + Datenleitungen).

2. Sicherstellen, dass alle USB-Einstellungen korrekt gesetzt sind (siehe Abschnitt oben) — insbesondere «USB CDC On Boot: Enabled».

3. Sketch → Hochladen (Strg+U). Die IDE erkennt den COM-Port automatisch und löst den Reset in den Bootloader-Modus aus.

Serial Monitor / Debugging

Baudrate: 115200. Nach dem Start gibt die Firmware auf dem seriellen Monitor den Initialisierungsablauf aus (Beispiel mit gespeichertem Heimnetz):

```
[NixieClock] Booting...
[Boot] ── Startprotokoll (12 Eintraege) ──
[Boot] ... (Historie, siehe unten)
[Boot] Aktueller Start #12, Reset-Grund: POWERON
[Nixie] MCP23017 initialisiert.
[WiFi] AP gestartet: NixieClockCS IP: 192.168.4.1
[WiFi] Verbinde mit 'MeinWLAN'...
[WiFi] STA verbunden. IP: 192.168.1.42
[mDNS] Erreichbar als http://nixieclockcs.local
[NTP] Synchronisierung gestartet...
[IR] Empfänger gestartet auf Pin 48
[NixieClock] Bereit.
[Boot] ── Startprotokoll (13 Eintraege) ── (erneut, inkl. aktuellem Start)
[NTP] RTC synchronisiert.
```

Zwischen «AP gestartet» und «Verbinde mit …» blenden die Röhren nacheinander auf (ca. 1,2 s). Ohne gespeichertes Heimnetz entfallen die STA-, mDNS- und NTP-Zeilen; schlägt die Verbindung fehl, erscheint «[WiFi] STA-Verbindung fehlgeschlagen – nur AP aktiv.»

Startprotokoll (boot_diag.ino)

Die Firmware führt im NVS (Namespace "bootlog") ein dauerhaftes Protokoll der letzten 20 Starts: Reset-Grund, zuletzt erreichter Start-Schritt und Zeitpunkt seit Start. Es übersteht auch einen vollständigen Spannungsverlust. Damit lassen sich Fehlstarts auswerten, die ohne angeschlossenen PC auftreten: Uhr am betroffenen Netzteil betreiben, danach am PC starten und die Ausgabe lesen. Weil der USB-Monitor die erste Ausgabe nach einem Reset oft verpasst, wird das Protokoll nach «Bereit.» ein zweites Mal ausgegeben.

```
[Boot] #6 Reset: BROWNOUT zuletzt: Funk/AP gestartet bei 307 ms
[Boot] #7 Reset: INT_WDT zuletzt: setup fertig bei 1523 ms
[Boot] #8 Reset: POWERON zuletzt: laeuft 10 min bei 600000 ms
```

Lesart: «Reset» gibt an, wodurch DIESER Start ausgelöst wurde, also wie der vorherige Start geendet hat. «zuletzt» zeigt, wie weit dieser Start gekommen ist, bevor der nächste Reset kam. Der letzte Eintrag ist der laufende Start. Start-Schritte in Reihenfolge: setup begonnen → Einstellungen geladen → MCP23017 init → Funk/AP gestartet → Roehren aufgeblendet → Heimnetz fertig → Web-Server gestartet → setup fertig → laeuft 10 s → laeuft 60 s → laeuft 10 min.

Reset-Grund	Bedeutung	Typische Ursache
POWERON	Normales Einschalten	Stromversorgung war ganz weg (Ein-/Ausstecken)
BROWNOUT	Unterspannung erkannt	Netzteil/Kabel liefert die Last nicht, Spannung bricht ein
PWR_GLITCH	Kurzer Spannungseinbruch	wie BROWNOUT
INT_WDT / TASK_WDT	Watchdog hat zugeschlagen	Firmware hängt; zusammen mit BROWNOUT meist ebenfalls Unterspannung (Flash-Lesefehler)
PANIC	Absturz (Exception)	Firmware-Fehler – Stacktrace im Serial Monitor beachten
SW / USB	Neustart per Software bzw. über USB	Flashen, Neustart aus der IDE

Häufige Probleme

Symptom	Ursache	Lösung
Röhren leuchten nicht	HV-MOD liefert keine Spannung	5-V-Versorgung und HV-MOD prüfen; Spannungsmessung an J4
Uhr startet nur an manchen Netzteilen, blinkt, bleibt stehen oder Röhren blitzen auf	Stromaufnahme zu hoch → Unterspannung (BROWNOUT)	Startprotokoll prüfen; Stromaufnahme messen (Richtwert ~760 mA mit 6 Röhren); Anodenwiderstände R65–R70 müssen 22 kΩ haben
Ghosting / Doppelziffern	I²C-Fehler, MCP nicht antwortend	I²C-Bus prüfen (Pull-ups R1/R3 auf dem Logic Board, Lötstellen SDA/SCL, Steckverbinder J3); Takt bei 100 kHz belassen
Web-UI nicht erreichbar	WiFi AP nicht gestartet	Serial Monitor prüfen; auf 192.168.4.1 verbinden
Uhrzeit falsch nach Reset	RTC leer / Batterie schwach	Zeit über Web-UI stellen; CR2032 BT1 prüfen
Kompilierungsfehler JSON	ArduinoJson v7 installiert	ArduinoJson auf v6.x downgraden
IR-Fernbedienung reagiert nicht	Code noch nicht angelernt	Im Web-UI unter IR-Fernbedienung Codes anlernen
ESP32 nach erstem Flash nicht mehr erreichbar, kein COM-Port	Mit «USB CDC On Boot: Disabled» kompiliert →	GPIO0-Testpunkt auf GND halten, USB neu einstecken → ROM-Bootloader → mit korrekten

	Firmware schaltet CDC ab	Einstellungen neu flashen